Sistemi di Calcolo 2: Programma Completo

Appunti Riorganizzati per lo Studio

Indice

1 Processi e Thread **3**
1.1 Creazione e Gestione dei Processi . . . 3
1.1.1 Casi particolari di terminazione (Zombie e Orfani) . . . 3
1.2 Thread e Multithreading . . . 3
1.2.1 Vantaggi e Stati dei Thread . . . 4
1.3 Tipologie di Thread: ULT e KLT . . . 4
1.4 Programmazione con PThread (POSIX) . . . 4
1.5 Architetture Parallele e SMP . . . 5

2 Concorrenza: Mutua Esclusione e Sincronizzazione **5**
2.1 Terminologia Fondamentale . . . 6
2.2 Difficoltà e Responsabilità del Sistema Operativo . . . 6
2.3 Supporto Hardware alla Mutua Esclusione . . . 6
2.3.1 Implementazione Hardware: La variabile `bolt` . . . 7
2.3.2 Approfondimento: Istruzioni Hardware Atomiche . . . 7
2.4 Meccanismi Software per la Concorrenza . . . 8
2.5 Dettaglio su Semafori e Monitor . . . 9
2.5.1 I Semafori . . . 9
2.5.2 I Monitor . . . 9
2.6 Message Passing . . . 9
2.7 Problemi Classici della Sincronizzazione . . . 9

3 Deadlock e Starvation **10**
3.1 Tipologie di Risorse e Condizioni per il Deadlock . . . 10
3.2 Approcci per la Gestione del Deadlock . . . 10
3.2.1 1. Prevenzione (Prevention) . . . 10
3.2.2 2. Evitamento (Avoidance) - L'Algoritmo del Banchiere . . . 11
3.2.3 3. Individuazione e Recupero (Detection and Recovery) . . . 11
3.3 Algoritmi Software per la Mutua Esclusione . . . 11
3.3.1 Algoritmo di Dekker . . . 12
3.3.2 Algoritmo di Dijkstra . . . 12
3.3.3 Algoritmo del Panettiere (Lamport's Bakery Algorithm, 1975) . . . 12

4 Comunicazione Inter Processo (IPC) **12**
4.1 1. Memoria Condivisa (Shared Memory) . . . 13
4.2 2. Scambio di Messaggi (Message Passing) . . . 13
4.3 Pipes e Named-Pipes (FIFOs) . . . 13
4.3.1 Pipe Standard (Anonime) . . . 13
4.3.2 Named-Pipes (FIFOs) . . . 14
4.4 Implementazione e Descrittori in UNIX . . . 14
4.4.1 Chiusura dei Descrittori ed Errori . . . 14

5 Socket e Reti **14**
5.1 Le Socket . . . 15
5.1.1 Indirizzamento e Porte . . . 15
5.2 Primitive di Comunicazione (Berkeley UNIX) . . . 15
5.2.1 Primitive TCP . . . 15
5.2.2 Primitive UDP . . . 16
5.3 Strutture Dati e Funzioni in C . . . 16
5.3.1 Funzioni di Utilità e Conversione . . . 16
5.4 Interazione Hardware: Il Network Adaptor . . . 16
5.4.1 Trasferimento Dati (Memoria ↔ Adaptor) . . . 17

6 Sistemi Distribuiti e IoT **17**
6.1 Perché usare i Sistemi Distribuiti? . . . 17
6.1.1 Sfide e Problematiche . . . 17
6.2 Architetture: Client-Server, Peer-to-Peer e SOA . . . 18
6.2.1 Il Middleware e le RPC . . . 18
6.3 Internet of Things (IoT) . . . 18

7 Sicurezza Informatica **18**
7.1 Obiettivi della Sicurezza . . . 18
7.2 Minacce, Vulnerabilità e Attacchi . . . 19
7.3 Malware . . . 19
7.4 Denial of Service e DDoS . . . 20
7.5 Il problema del Buffer Overflow . . . 20
7.5.1 Contromisure al Buffer Overflow . . . 20
7.6 Autenticazione e Gestione delle Password . . . 21
7.7 Crittografia . . . 21
7.7.1 Crittografia Simmetrica . . . 21
7.7.2 Crittografia Asimmetrica . . . 21
7.8 Integrità dei messaggi . . . 22
7.9 Controllo degli Accessi . . . 22
7.9.1 DAC – Discretionary Access Control . . . 22
7.9.2 MAC – Mandatory Access Control . . . 22
7.9.3 RBAC – Role-Based Access Control . . . 23
7.9.4 Confronto tra DAC, MAC e RBAC . . . 23
7.10 Firewall . . . 23
7.11 Honeypot . . . 24
7.12 Principi generali di progettazione sicura . . . 24

8 Algoritmo di Dekker (2 Processi) **24**
8.1 Soluzione Definitiva (Dekker) . . . 25

9 Algoritmo di Dijkstra (Generalizzazione a N processi) **27**

10 Algoritmo del Panettiere (Lamport's Bakery) **27**
10.1 La Soluzione Definitiva di Lamport . . . 28

1 Processi e Thread

Ogni programma è fondamentalmente un processo in esecuzione. Esso comprende diverse componenti fondamentali: il codice, i dati e i suoi attributi. Gli attributi principali di un processo includono l'identificatore (PID), lo stato, la priorità, i puntatori, lo stato dei registri della CPU, lo stato dell'I/O e l'indirizzo della prossima istruzione da eseguire.

Tutte queste informazioni vengono salvate all'interno del **Process Control Block (PCB)**. Il PCB viene creato e gestito direttamente dal Sistema Operativo (SO), il quale può così supportare e alternare l'esecuzione di più processi contemporaneamente. Il SO garantisce a ciascun processo tutte le risorse necessarie e fornisce i meccanismi affinché più processi possano comunicare fra di loro.

1.1 Creazione e Gestione dei Processi

All'avvio della macchina, il *Bootstrap Program* inizializza la memoria, i controller dei device e i registri della CPU. Successivamente, carica il Sistema Operativo in memoria e lo avvia. Il SO crea il primissimo processo, chiamato `init`, e si mette in attesa di eventi come interrupt hardware o software (trap).

Per la gestione dei processi nei sistemi UNIX si utilizzano specifiche System Call:

- **fork**: Permette a un processo (padre) di creare un nuovo processo (figlio). All'inizio, padre e figlio sono quasi identici: il figlio eredita una copia esatta della memoria, dei registri della CPU, dei file aperti e del PCB (eccetto per l'ID del processo). La funzione restituisce `-1` in caso di errore, `0` al processo figlio e l'identificatore (PID) del figlio al processo padre. Questo permette di inserire dei controlli nel codice per far biforcare l'esecuzione.

- **exec**: Utilizzata all'interno del codice del figlio per lanciare un nuovo programma. Questa chiamata sovrascrive interamente lo spazio di memoria del processo figlio con il nuovo programma.

- **wait e waitpid**: Un processo padre può decidere di sospendersi in attesa che il figlio termini il suo compito. La `wait` attende la terminazione del primo figlio disponibile e ne restituisce il PID (o `-1` se non ci sono figli). Passando il puntatore di una variabile alla `wait`, è possibile catturare lo stato di uscita del figlio. La `waitpid`, invece, permette di attendere un figlio specifico.

- **exit**: Invocata alla fine dell'esecuzione. Prende come parametro uno stato per indicare se il processo è terminato correttamente. Si occupa di chiudere i file, le connessioni e deallocare la memoria.

1.1.1 Casi particolari di terminazione (Zombie e Orfani)

Se un processo termina ma ha ancora dei figli attivi, questi vengono "adottati" dal processo `init`. Se invece un processo figlio termina mentre il padre è ancora vivo ma non ha ancora effettuato una `wait`, il figlio entra nello stato di **zombie**: rimane nel sistema solo per mantenere il risultato (exit status) da restituire al padre non appena quest'ultimo lo richiederà. Altrimenti, una volta letto lo stato, il processo muore definitivamente.

1.2 Thread e Multithreading

Un processo è caratterizzato da due elementi: le risorse richieste (dimensione spaziale) e il cammino di esecuzione (dimensione temporale). Della parte temporale si occupano i **thread**.

Un SO può supportare diverse combinazioni: un processo con un singolo thread, più processi monothread, un singolo processo multithread, o più processi multithread. Il **Multithreading**

è proprio l'abilità del SO di supportare più thread contemporaneamente all'interno dello stesso processo. In questo scenario, il processo mantiene un singolo spazio di indirizzamento virtuale e l'accesso condiviso a file e risorse I/O.

Mentre un processo monothread possiede un solo PCB, un user stack e un kernel stack, un processo multithread possiede un PCB generale e, per ciascun thread, un **Thread Control Block (TCB)**, un user stack e un kernel stack separati. Ogni thread mantiene il proprio stato di esecuzione, il *context data* (salvato quando non è in esecuzione), e uno spazio statico per le variabili locali, pur condividendo la memoria globale del processo.

1.2.1 Vantaggi e Stati dei Thread

I benefici principali nell'uso dei thread includono:

- Rapidità di creazione e terminazione rispetto ai processi.
- Switch tra thread estremamente veloce.
- La comunicazione tra thread dello stesso processo non necessita l'invocazione del kernel.
- Possibilità di lavorare simultaneamente in background e in foreground.
- Esecuzione fluida: un thread può proseguire il calcolo mentre un altro è bloccato in attesa di risorse I/O.

Gli stati in cui può trovarsi un thread sono: **running**, **ready** e **blocked**. Un thread può crearne un altro, bloccarsi, sbloccarsi o terminare. Sospendere o terminare un intero processo significa sospendere o terminare a cascata tutti i thread che ne fanno parte.

Un tipico esempio di utilità è nelle **Remote Procedure Call (RPC)**: se un client deve fare due richieste al server remoto, anziché farle sequenzialmente attendendo il risultato di ciascuna, può creare un secondo thread per gestire la seconda richiesta in parallelo alla prima.

1.3 Tipologie di Thread: ULT e KLT

I thread possono essere implementati su due livelli:

- **User Level Thread (ULT)**: Il kernel non sa dell'esistenza dei thread, che sono gestiti interamente da una libreria a livello utente. Questo evita le perdite di tempo dovute ai cambi di contesto nel kernel e rende il codice portabile. Tuttavia, poiché il kernel vede un solo processo, assegnerà un solo core: i thread non gireranno mai in vero parallelo hardware, e se un thread si blocca, bloccherà l'intero processo.
- **Kernel Level Thread (KLT)**: Il kernel mantiene il contesto sia del processo che dei singoli thread e si occupa direttamente dello scheduling. Questo permette la vera esecuzione contemporanea su più core, anche se comporta un leggero calo di prestazioni dovuto al passaggio nel kernel per ogni operazione di scheduling.

Esistono anche approcci ibridi che combinano i vantaggi di entrambi i modelli.

1.4 Programmazione con PThread (POSIX)

Nei sistemi UNIX si utilizza la libreria **PThread** per implementare il multithreading senza il pesante overhead di una `fork`. Vi sono due schemi classici di programmazione:

1. **Manager/Worker**: Un thread principale crea altri thread delegando loro dei compiti e attende i risultati.

2. **Pipeline**: Il task principale viene diviso in sotto-task sequenziali lanciati contemporaneamente tramite thread separati.

Tutti i thread hanno accesso alla memoria globale del processo. Un codice è definito **thread-safe** quando più thread possono essere eseguiti concorrentemente senza interferire tra loro e senza corrompere i dati in memoria condivisa.

Le funzioni principali sono:

- `pthread_create`: Crea un nuovo thread, mettendolo allo stesso livello del creatore.
- `pthread_exit`: Termina il thread al completamento del suo lavoro e permette di passare un valore di ritorno (non libera le risorse automaticamente come una `exit` di sistema).
- `pthread_join`: Cattura il valore restituito dalla `pthread_exit` del thread atteso.
- `pthread_cancel`: Permette a un thread di forzare la terminazione di un altro thread.

Attenzione: L'esecuzione di una generica `exit` in un qualsiasi thread causa la morte dell'intero processo. Se il thread `main` termina, termina tutto. Per evitare questo, si usa `pthread_exit` alla fine del main, oppure si usa `pthread_detach` per scollegare i thread creati dalla dipendenza del main.

1.5 Architetture Parallele e SMP

Mentre in passato si lavorava su macchine sequenziali, oggi si sfruttano architetture parallele. La **Tassonomia di Flynn** categorizza i sistemi in:

- **SISD**: Singola istruzione su singolo dato.
- **SIMD**: Singola istruzione su dati multipli.
- **MISD**: Multiple istruzioni su singolo dato.
- **MIMD**: Multiple istruzioni su dati multipli.

L'approccio più comune oggi è il **Symmetric MultiProcessor (SMP)**. Una tipica architettura SMP è composta da memoria virtuale, I/O adapter e un set di processori (ciascuno con la propria cache L1 ed L2), tutti interconnessi tramite un bus condiviso. Questa struttura solleva problemi complessi riguardanti la pianificazione (scheduling), la sincronizzazione e la coerenza delle memorie condivise.

2 Concorrenza: Mutua Esclusione e Sincronizzazione

Un Sistema Operativo (SO) può gestire l'esecuzione concorrente in tre diversi scenari:

- **Multiprogrammazione**: più processi eseguiti su un singolo core.
- **Multiprocessi**: più processi eseguiti su più core.
- **Gestione distribuita**: più processi distribuiti su più macchine differenti.

La concorrenza si presenta tipicamente in tre contesti: nella condivisione del tempo fra applicazioni multiple, nella gestione di applicazioni strutturate (composte da più processi cooperanti) e nella struttura stessa del SO (che è a sua volta composto da processi e thread).

2.1 Terminologia Fondamentale

Per affrontare i problemi della concorrenza, è indispensabile definire i seguenti concetti:

- **Operazione atomica**: una sequenza di istruzioni che, una volta iniziata, non può essere interrotta.
- **Sezione critica**: una porzione di codice in cui si richiede l'accesso a risorse condivise, le quali non devono essere toccate da altri thread contemporaneamente.
- **Mutua esclusione**: la garanzia che, quando un processo si trova in una sezione critica per accedere a una risorsa condivisa, nessun altro processo possa entrare in una sezione critica per quella stessa risorsa.
- **Race Condition (Corsa alla risorsa)**: situazione in cui due o più thread/processi leggono e scrivono contemporaneamente in una memoria condivisa. Il risultato finale dipende in modo imprevedibile dalle tempistiche di esecuzione (timing), poiché l'ultimo processo a scrivere sovrascriverà i dati precedenti.
- **Deadlock**: blocco permanente in cui un processo attende il verificarsi di una condizione che non accadrà mai (es. due processi che attendono ciascuno che l'altro rilasci una risorsa).
- **Livelock**: situazione in cui due o più processi cambiano continuamente il proprio stato in risposta ai cambiamenti degli altri, senza però mai compiere un lavoro utile.
- **Starvation**: condizione in cui un processo è pronto a ripartire, ma viene continuamente ignorato dallo scheduler rimanendo inattivo, magari bloccando a sua volta altre risorse.

2.2 Difficoltà e Responsabilità del Sistema Operativo

Le difficoltà principali della concorrenza risiedono nella condivisione di memorie globali, nella gestione delle risorse da parte del SO e nell'estrema difficoltà di fare *debugging*. Gli errori di programmazione concorrente, infatti, portano a esecuzioni non deterministiche e irriproducibili: la velocità del processo non è prevedibile perché dipende dalle interrupt e dallo scheduling.

Il SO ha l'onere di conoscere tutti i processi in esecuzione e il loro stato, allocare/deallocare le risorse per ognuno, proteggere dati e risorse dalle interferenze e garantire che l'output finale sia del tutto indipendente dalla velocità di esecuzione dei singoli processi.

Quando i processi concorrenti competono per le stesse risorse (entrando in conflitto), si devono affrontare tre problemi fondamentali: garantire la **mutua esclusione**, evitare il **deadlock** e prevenire la **starvation**. La sezione critica, inoltre, deve sempre avere una durata di tempo finita.

2.3 Supporto Hardware alla Mutua Esclusione

Per garantire la mutua esclusione, si può intervenire a livello hardware in due modi principali:

1. **Disabilitazione degli Interrupt**: il processo in sezione critica disabilita gli interrupt del SO, impedendo che venga interrotto. *Svantaggi*: l'efficienza del sistema peggiora drasticamente e questo approccio non funziona su architetture multiprocessore.
2. **Istruzioni Macchina Speciali**: istruzioni eseguite in modo nativamente atomico.
    - **Compare & Swap**: confronta un valore in memoria con un valore di test. Se uguali, il valore in memoria viene scambiato con uno nuovo. Disponibile su X86, IA64, SPARC, IBM.
    - **Exchange**: scambia il valore contenuto in un registro con un valore in memoria. Disponibile in Pentium e Itanium (IA64).

2.3.1 Implementazione Hardware: La variabile `bolt`

Utilizzando l'istruzione Compare & Swap, si utilizza una variabile globale `bolt` inizializzata a 0. Il primo thread che legge `bolt` = 0 la imposta a 1 ed entra nella sezione critica. I thread successivi leggeranno `bolt` = 1, la *compare* fallirà e rimarranno in attesa in un ciclo. Quando il primo thread esce, rimette `bolt` a 0.

- **Vantaggi**: applicabile a qualsiasi numero di processi su uno o più processori con memoria condivisa; molto semplice; supporta diverse sezioni critiche usando più variabili.
- **Svantaggi**: causa *busy-waiting* (il processo in attesa consuma tempo CPU ciclando a vuoto); rischio di *starvation* per i processi in attesa; rischio di *deadlock* se si utilizzano priorità rigide (un processo a bassa priorità in sezione critica potrebbe essere interrotto ma non rilasciare la risorsa a un processo ad alta priorità in busy-waiting).

2.3.2 Approfondimento: Istruzioni Hardware Atomiche

Per superare i limiti della disabilitazione degli interrupt nei sistemi multiprocessore, le architetture moderne offrono speciali istruzioni macchina. Queste istruzioni operano a livello hardware eseguendo letture e scritture in un unico ciclo indivisibile (operazione atomica). Le due varianti principali discusse nel corso sono **Compare & Swap** ed **Exchange**.

1. L'istruzione Compare & Swap (CAS) Disponibile su architetture come x86, IA64, SPARC e IBM, questa istruzione confronta un valore presente in memoria con un valore di test (atteso). Se i due valori coincidono, il valore in memoria viene aggiornato atomicamente con un nuovo valore. In ogni caso, l'istruzione restituisce il vecchio valore che era presente in memoria prima dell'operazione.

```
int compare_and_swap(int *word, int testval, int newval) {
    int oldval = *word;
    if (oldval == testval) {
        *word = newval;
    }
    return oldval;
}
```

Utilizzo per la mutua esclusione: Si definisce una variabile globale condivisa, ad esempio `bolt`, inizializzata a `0`. Quando un processo vuole entrare nella sezione critica, esegue un ciclo *while* basato sull'istruzione CAS: cerca di verificare che `bolt` sia a `0` e, contestualmente, lo imposta a `1`. Se `bolt` era già a `1` (risorsa occupata), la funzione restituisce `1` e il processo rimane intrappolato nel ciclo (busy-waiting) finché la risorsa non viene liberata.

2. L'istruzione Exchange (XCHG) Disponibile in processori come Pentium e Itanium (IA64), questa istruzione scambia, in modo strettamente atomico, il contenuto di un registro della CPU con il contenuto di una specifica locazione di memoria.

```
void exchange(int *register_val, int *memory_val) {
    int temp = *memory_val;
    *memory_val = *register_val;
    *register_val = temp;
}
```

Utilizzo per la mutua esclusione: Anche qui si usa un flag globale `bolt` a `0`. Il processo dichiara una variabile locale (registro) chiamata `key` inizializzata a `1`. Il processo continua a eseguire ciclicamente l'`exchange` tra la sua `key` e `bolt`. Riuscirà a entrare nella sezione critica solo quando `bolt` diventerà `0`, poiché l'istruzione metterà `0` in `key` e `1` in `bolt`, bloccando l'accesso agli altri. All'uscita dalla sezione critica, il processo imposterà semplicemente `bolt = 0`.

Vantaggi e Svantaggi dell'Approccio Hardware

- **Vantaggi:** Sono applicabili su un qualsiasi numero di processi e processori che condividono la memoria principale. Inoltre, sono di semplice implementazione e permettono di gestire facilmente diverse sezioni critiche dichiarando più variabili di controllo distinte.
- **Svantaggi:**
  - *Busy-waiting:* Il processo in attesa continua a consumare attivamente tempo prezioso del processore eseguendo cicli a vuoto.
  - *Starvation:* Quando un processo lascia la sezione critica, se ci sono più processi in attesa non c'è una politica FIFO a stabilire chi entrerà; la scelta dipende solo da chi esegue l'istruzione per primo in quel micro-istante.
  - *Deadlock (inversione di priorità):* Se un processo a bassa priorità possiede la risorsa e viene interrotto da uno ad alta priorità che necessita della medesima risorsa, quest'ultimo inizierà un busy-waiting infinito, monopolizzando la CPU e impedendo al processo a bassa priorità di finire il lavoro e rilasciare il `bolt`.

2.4 Meccanismi Software per la Concorrenza

Oltre all'hardware, il SO offre meccanismi software per la sincronizzazione:

- **Semaforo**: variabile intera usata per la segnalazione. Permette solo tre operazioni: `initialize`, `decrement` e `increment`.
- **Semaforo binario**: semaforo che assume solo valori 0 o 1.
- **Mutex**: simile al semaforo binario, ma impone che il processo che ha bloccato il mutex (valore a 0) debba essere obbligatoriamente lo stesso processo a sbloccarlo (valore a 1).
- **Condition Variable**: costrutto che blocca un processo finché una determinata condizione logica non si avvera.
- **Monitor**: costrutto di programmazione che contiene metodi e variabili. Se un metodo del monitor viene chiamato da un processo, nessun altro metodo può essere eseguito finché il primo non termina (mutua esclusione intrinseca).
- **Event Flag**: un processo rimane bloccato finché una specifica sequenza di bit non viene settata.
- **Mailbox / Messaggi**: scambio di messaggi espliciti tra processi come meccanismo combinato di sincronizzazione e scambio dati.
- **Spinlocks**: cicli infiniti eseguiti in attesa della modifica di una variabile (forma di busy-waiting controllato).

2.5 Dettaglio su Semafori e Monitor

2.5.1 I Semafori

Inizializzati a un valore non negativo, vengono manipolati solo tramite `semWait` (decremento, usato per entrare in sezione critica) e `semSignal` (incremento, usato per uscire). Non è possibile sapere a priori se un processo si bloccherà effettuando la `semWait`, né si può prevedere quale tra due processi concorrenti verrà bloccato.

- **Strong Semaphore**: i processi in coda escono in ordine rigoroso FIFO (il processo bloccato da più tempo entra per primo).
- **Weak Semaphore**: l'ordine di risveglio dei processi dalla coda non è specificato.

Le funzioni `semWait` e `semSignal` devono essere intrinsecamente atomiche. Per implementare questa atomicità si possono usare le istruzioni hardware (Compare & Swap per proteggere il semaforo stesso), inibire gli interrupt, oppure algoritmi software puri come Dekker o Peterson.

2.5.2 I Monitor

Rendono l'utilizzo della mutua esclusione molto più semplice e sicuro rispetto ai semafori manuali (sebbene in linguaggio C non ci sia una libreria ufficiale per essi). All'interno del Monitor risiedono variabili locali visibili solo ai suoi metodi. La sincronizzazione interna avviene tramite variabili di condizione, modificate con primitive `cwait(c)` e `csignal(c)` riferite alla specifica condizione `c`.

2.6 Message Passing

Essenziale nei sistemi distribuiti (che non condividono una memoria centrale). Permette sia la sincronizzazione che lo scambio di dati. Le primitive base sono `send(destination, message)` e `receive(source, message)`.

Quando si invoca una `receive`, se un messaggio è pronto si prosegue, altrimenti ci si blocca (o si abbandona il tentativo, a seconda dell'implementazione). La comunicazione definisce diverse combinazioni logiche:

- **Blocking Send, Blocking Receive**: mittente e destinatario rimangono entrambi bloccati finché il messaggio non è stato fisicamente consegnato (sincronizzazione stretta).
- **Nonblocking Send, Blocking Receive**: è la combinazione più comune. Il mittente lancia il messaggio e prosegue subito (utile per inviare a destinazioni multiple rapidamente), mentre il ricevente si blocca finché non c'è un dato da leggere.
- **Nonblocking Send, Nonblocking Receive**: nessuno dei due si blocca mai.

L'indirizzamento può essere **diretto** (mittente specifica il destinatario) o **indiretto** (i messaggi vengono inviati in una **Mailbox** condivisa, strutturata come coda, e i destinatari li prelevano da lì). Ogni messaggio ha una struttura fissa: tipo del messaggio, ID source/destination, lunghezza, controlli e contenuto utile.

2.7 Problemi Classici della Sincronizzazione

- **Produttore / Consumatore**: uno o più produttori inseriscono dati in un buffer e uno o più consumatori li leggono. Solo un processo alla volta può accedere al buffer. Il problema richiede che i consumatori non leggano se il buffer è vuoto e, nel caso di *buffer limitato*, che i produttori non scrivano se il buffer è pieno. Spesso si usano due semafori separati per gestire la disponibilità di lettura e scrittura.

- **Readers / Writers (Lettori / Scrittori)**: vi è un'area di memoria condivisa. È consentito a più *lettori* di leggere in contemporanea senza limiti. Tuttavia, la scrittura deve essere rigidamente esclusiva: un solo *scrittore* alla volta può accedere, e mentre lo scrittore opera, non è consentita l'azione di nessun lettore.

3 Deadlock e Starvation

Il **Deadlock** (stallo) si verifica quando un processo bloccato attende un segnale di sblocco da un altro processo, il quale a sua volta è bloccato in attesa del primo. È una situazione permanente, non ha soluzione autonoma e deve quindi essere gestita o evitata dal Sistema Operativo.

3.1 Tipologie di Risorse e Condizioni per il Deadlock

Per comprendere il deadlock, bisogna prima classificare le risorse in due categorie:

- **Risorse Riutilizzabili**: Possono essere utilizzate in sicurezza da un solo processo alla volta e non vengono distrutte dopo l'uso (es. processore, dispositivi di I/O, database, file, semafori).
- **Risorse Consumabili**: Vengono create e poi distrutte dopo l'utilizzo (es. interrupt, segnali, messaggi).

Affinché si verifichi un deadlock, devono sussistere contemporaneamente **quattro condizioni necessarie** (le prime tre da sole non sono sufficienti):

1. **Mutua esclusione**: I processi richiedono l'uso esclusivo delle risorse. Se la risorsa fosse condivisibile contemporaneamente, non ci sarebbe attesa.
2. **Hold-and-wait (Trattieni e attendi)**: Un processo mantiene bloccate delle risorse già acquisite mentre è in attesa che gliene vengano assegnate di nuove.
3. **No preemption (Nessuna prelazione)**: Il Sistema Operativo non può sottrarre forzatamente una risorsa a un processo che la detiene; deve attendere il rilascio volontario.
4. **Circular wait (Attesa circolare)**: Esiste una catena chiusa di processi in cui ognuno detiene almeno una risorsa richiesta dal processo successivo nella catena.

3.2 Approcci per la Gestione del Deadlock

Esistono tre strategie principali per affrontare il problema del deadlock.

3.2.1 1. Prevenzione (Prevention)

L'obiettivo è progettare il sistema in modo da eliminare alla radice una delle quattro condizioni necessarie.

- **Prevenzione Indiretta** (mira alle prime tre condizioni):
  - *Mutua esclusione*: Spesso non può essere eliminata perché intrinseca in alcune risorse, ma il SO può cercare di renderla meno stringente dove possibile.
  - *Hold-and-wait*: Si impone a un processo di richiedere tutte le risorse necessarie in un'unica volta all'avvio. *Svantaggi:* genera lunghe attese iniziali e spesso è impossibile sapere in anticipo l'esatto fabbisogno di risorse.
  - *No preemption*: Il SO viene autorizzato a revocare forzatamente una risorsa. Richiede un rigido sistema di priorità e la capacità di salvare e ripristinare lo stato del processo prelazionato.

- **Prevenzione Diretta** (mira alla quarta condizione):
  - *Circular wait*: Si definisce un ordinamento lineare (gerarchia) universale per tutte le risorse. Un processo può richiedere nuove risorse solo seguendo l'ordine prestabilito (es. se ho la risorsa 2, posso chiedere solo la 3 o la 4). *Svantaggi:* poco flessibile e vincola fortemente la programmazione.

3.2.2 2. Evitamento (Avoidance) - L'Algoritmo del Banchiere

In questo approccio, il SO compie scelte dinamiche basate sullo stato attuale e sulle future richieste previste. Si valuta ogni singola richiesta di allocazione per capire se porterà a un deadlock. Definiamo **Safe State (Stato Sicuro)** una condizione in cui esiste almeno una sequenza di esecuzione che permette a tutti i processi di terminare senza incorrere in deadlock.

- **Process Initiation Denial**: Un processo non viene nemmeno avviato se le sue richieste massime, sommate a quelle degli altri, superano le risorse totali disponibili nel sistema.
- **Resource Allocation Denial (Algoritmo del Banchiere di Dijkstra)**: Meno restrittivo. Quando un processo richiede una risorsa, il SO "simula" l'allocazione. Tramite vettori e matrici che tracciano risorse disponibili, allocate e massime richieste (`Claim`, `Alloc`, `Available`), il SO verifica se il nuovo stato simulato è ancora un *Safe State*. Se lo è, la risorsa viene concessa; altrimenti, il processo viene sospeso in attesa.
- *Svantaggi dell'Evitamento:* Richiede di conoscere in anticipo il limite massimo di risorse richieste; funziona solo per processi indipendenti (senza sincronizzazione tra loro); il numero di risorse disponibili deve essere rigorosamente fisso.

3.2.3 3. Individuazione e Recupero (Detection and Recovery)

Non si previene e non si evita: si concedono le risorse ogni volta che è possibile, ma si controlla periodicamente lo stato del sistema per scovare deadlock in corso.

- **Individuazione**: Si esegue un protocollo (simile all'Algoritmo del Banchiere) che tenta iterativamente di "marcare" i processi che possono terminare. Se alla fine del controllo ci sono processi non marcati, questi sono in deadlock. Eseguire questo controllo a ogni singola allocazione individua subito il problema ma è pesantissimo per la CPU.
- **Recupero**: Una volta rilevato il deadlock, le soluzioni possibili sono:
  1. Uccidere tutti i processi in deadlock (la più comune e drastica).
  2. Ripristinare il sistema a un *checkpoint* precedente sicuro (rollback), anche se il deadlock potrebbe ripresentarsi.
  3. Uccidere i processi bloccati uno alla volta, rieseguendo l'algoritmo di individuazione finché il ciclo non si spezza.
  4. Usare la preemption per sottrarre forzatamente risorse ai processi bloccati finché non si sblocca la situazione.

3.3 Algoritmi Software per la Mutua Esclusione

Se non si dispone di supporto hardware o di primitive del SO (come i semafori), la mutua esclusione può essere garantita puramente tramite software usando una memoria centrale condivisa (pur sapendo che read/write base potrebbero causare problemi). Prima di arrivare a soluzioni funzionanti, tentativi ingenui includono l'uso di una singola variabile `turn` (causa busy-waiting e stretta alternanza) o flag multipli (causano *livelock* in cui i processi si cedono continuamente il passo senza avanzare).

3.3.1 Algoritmo di Dekker

Risolve il problema della mutua esclusione per esattamente **2 processi**. Utilizza un array di booleani (`flag[2]`) per indicare l'intenzione di entrare in sezione critica e una variabile intera (`turn`) per risolvere i conflitti se entrambi alzano il flag simultaneamente.

3.3.2 Algoritmo di Dijkstra

Generalizza l'idea di Dekker per un numero $N > 2$ di processi. Utilizza array condivisi (`interested[N]` e `passed[N]`) e una variabile di turno `k`.

- *Pro:* Garantisce la mutua esclusione ed evita il deadlock.
- *Contro:* **Non garantisce l'assenza di starvation**. Un processo con priorità minore potrebbe rimanere bloccato nel ciclo *while* consumando CPU (busy-waiting) mentre altri processi più veloci continuano ad accaparrarsi il turno. Inoltre, necessita che le operazioni di lettura e scrittura in memoria siano assolutamente atomiche.

3.3.3 Algoritmo del Panettiere (Lamport's Bakery Algorithm, 1975)

Sviluppato da Leslie Lamport, si basa sul concetto dei "numeretti" presi in panetteria per fare la fila. Ogni processo che vuole entrare nella sezione critica prende un numero (maggiore di tutti i numeri attualmente in uso). Entrerà in sezione critica il processo con il numero più basso. In caso di parità di numero, passa il processo con l'identificatore (PID) più basso.

- **Caratteristiche distintive:** Rispetto a Dijkstra, elimina la necessità di operazioni atomiche di read/write. I processi comunicano tramite due array condivisi: `choosing` (booleano, indica se il processo sta estraendo il numero) e `number` (intero, il ticket estratto).
- Nessun processo può effettuare scritture concorrenti sulla stessa locazione di memoria: ognuno scrive unicamente nel proprio indice dell'array e si limita a leggere gli indici degli altri. I tempi di esecuzione sono completamente svincolati.

4 Comunicazione Inter Processo (IPC)

Di base, un processo può accedere solo alla propria area di memoria riservata, mentre il Kernel ha accesso a tutto. I processi all'interno di un sistema possono essere:

- **Indipendenti**: L'esecuzione di un processo non influisce in alcun modo su quella di un altro.
- **Cooperativi**: Il risultato di un processo dipende dall'esecuzione o dai dati di un altro.

Le ragioni principali per cui si sceglie di far cooperare due o più processi includono:

1. **Modularità**: È preferibile suddividere un programma complesso in task più semplici e indipendenti.
2. **Convenienza**: Lavorare su più processi può risultare architetturalmente migliore.
3. **Velocità di calcolo**: Se una parte del programma è in attesa (ad esempio di un'operazione di I/O), il resto del programma può proseguire l'esecuzione senza bloccarsi.
4. **Scambio di informazioni**: Necessità di condividere dati e risultati parziali.

Come comunicano i processi se non hanno memoria in comune? Utilizzare un file fisico su disco sarebbe una soluzione estremamente lenta e poco pratica. I Sistemi Operativi offrono quindi due meccanismi principali di *Inter-Process Communication* (IPC): la **Memoria Condivisa** e lo **Scambio di Messaggi**.

4.1 1. Memoria Condivisa (Shared Memory)

Nel caso della memoria condivisa, il Kernel assegna un'area di memoria specifica che può essere acceduta direttamente da entrambi i processi.

- **Vantaggi**: Una volta che l'area è stata assegnata, l'accesso è rapidissimo perché i processi possono leggere e scrivere in questa zona senza dover richiamare il Kernel a ogni operazione.
- **Svantaggi e Gestione**: Poiché più processi possono leggere e scrivere contemporaneamente, è assolutamente necessario implementare un meccanismo di sincronizzazione (come i **semafori**) per impedire l'accesso simultaneo e le relative *race condition*.
- **Implementazione**: Nei sistemi UNIX si utilizza una serie di funzioni fornite dalla **POSIX Shared Memory API**.

4.2 2. Scambio di Messaggi (Message Passing)

Nel Message Passing, i messaggi vengono scambiati transitando fisicamente attraverso il Kernel. I messaggi inviati vengono salvati in una coda gestita dal SO.

- **Svantaggi**: Un processo scrive nella coda e l'altro legge, ma entrambi devono effettuare una System Call al Kernel per ogni singola operazione di I/O sui messaggi, causando una notevole perdita di tempo (overhead).
- **Primitive Fondamentali**: Servono almeno due funzioni base: `send` e `receive`. Entrambe prendono come parametro obbligatorio il *messaggio*, e come parametri facoltativi il *destinatario* (nella send) e il *mittente* (nella receive).
- **Struttura del Messaggio**: Un messaggio ha un formato rigidamente stabilito che contiene: tipo del messaggio, ID destinatario, ID mittente, lunghezza, informazioni di controllo e il contenuto vero e proprio. (N.B. Nello standard POSIX, le header dei messaggi non contengono esplicitamente l'ID del mittente e del destinatario).
- **Code**: Generalmente i messaggi vengono gestiti con una politica **FIFO** (First In, First Out), ma è possibile implementare logiche basate sulle priorità.

4.3 Pipes e Named-Pipes (FIFOs)

Il Message Passing nei sistemi UNIX viene tipicamente realizzato attraverso le **Pipes**. A livello di Sistema Operativo, una pipe non è altro che un buffer di dimensione prestabilita gestito in memoria centrale.

4.3.1 Pipe Standard (Anonime)

- Attivano un collegamento **monodirezionale** tra due processi **relazionati** (uno dei due deve essere stato generato tramite una `fork` dall'altro, ovvero padre e figlio).
- Permettono di comunicare come se si stesse accedendo a un file sequenziale: una volta che un'informazione viene letta dalla pipe, essa viene eliminata definitivamente dal buffer.
- Quando i processi terminano, la pipe viene distrutta autonomamente dal sistema.

4.3.2 Named-Pipes (FIFOs)

- Permettono la comunicazione anche tra processi **non relazionati** (che non hanno vincoli di parentela).
- Sono collegamenti **bidirezionali**.
- Non si chiudono autonomamente al termine dei processi, ma persistono nel sistema finché non vengono eliminate esplicitamente.

4.4 Implementazione e Descrittori in UNIX

Nei sistemi Unix, l'uso delle pipe avviene attraverso la nozione di **descrittore di file** (file descriptor).

- **Creazione**: Si utilizza la system call `pipe(int fd[2])`. Questa funzione prende come argomento un array di due interi e lo popola con due descrittori:
  - `fd[0]`: Descrittore aperto in sola **lettura**.
  - `fd[1]`: Descrittore aperto in sola **scrittura**.
- **Lettura/Scrittura (`read` e `write`)**: Se si prova a leggere da una pipe vuota, il processo lettore si blocca in attesa che si riempia. Se si prova a scrivere in una pipe piena, il processo scrittore si blocca in attesa che il lettore liberi spazio nel buffer.
- **Limiti e Atomicità**: In Linux, una pipe ha una dimensione massima equivalente a **4096 byte** (16 pagine di memoria). Una scrittura di dimensione minore di 4096 byte è garantita come **atomica**. Se si cerca di scrivere un quantitativo di dati maggiore della capienza del buffer, il processo si bloccherà e c'è il rischio che altri processi si intromettano scrivendo dati, frammentando così il messaggio originale.

4.4.1 Chiusura dei Descrittori ed Errori

Le pipe sono dispositivi logici limitati.

- **Fine scrittura**: Una pipe viene considerata "finita" quando nessun descrittore in scrittura (`fd[1]`) è più aperto nel sistema. In questo caso, una chiamata `read` da parte del lettore non si blocca, ma restituisce semplicemente `0` (EOF - End Of File).
- **Fine lettura (Errore SIGPIPE)**: Al contrario, se tutti i descrittori in lettura (`fd[0]`) sono stati chiusi e un processo tenta comunque di scriverci dentro con una `write`, il Kernel invierà al processo scrittore un segnale di errore fatale chiamato `SIGPIPE`.
- **Prevenzione Deadlock**: Per evitare blocchi, è fondamentale **chiudere i descrittori non appena non servono più**. In un classico schema padre-figlio, subito dopo la fork, il processo lettore deve chiudere il proprio lato di scrittura (`fd[1]`), e il processo scrittore deve chiudere il proprio lato di lettura (`fd[0]`).

5 Socket e Reti

Lo scambio di dati tra entità all'interno di sistemi distribuiti o connessi in rete può essere molto complesso. Per questo motivo, la comunicazione è regolata da **Protocolli**, definiti da tre elementi chiave: *semantica*, *sintassi* e *timing*. Il mittente deve informare la rete riguardo l'identità del destinatario (o attivare un collegamento diretto) e assicurarsi che questo sia pronto a ricevere e salvare i dati. Tutta la comunicazione si basa sulla collaborazione di vari moduli organizzati in **livelli di rete**.

5.1 Le Socket

Una **Socket** è l'astrazione software che permette la comunicazione tra un processo *Client* e un processo *Server*. Esistono tre tipologie principali di socket:

- **Stream Socket (TCP)**: Connessione affidabile, orientata alla connessione (garantisce l'ordine e la consegna dei pacchetti).
- **Datagram Socket (UDP)**: Connessione non affidabile, senza connessione stabilita a priori (più veloce ma senza garanzie di consegna).
- **Raw Socket**: Consente l'accesso diretto ai livelli più bassi del protocollo di rete (usata per scopi diagnostici o di sicurezza).

5.1.1 Indirizzamento e Porte

Per identificare univocamente una socket in rete si utilizza l'abbinamento tra l'**Indirizzo IP** dell'host e un **Numero di Porta**. Quando il client manda una richiesta, spesso utilizza una porta temporanea per contattare il server su una porta standard (es. 80 per HTTP). Il server risponderà, ma non necessariamente sulla porta iniziale del server, per poter gestire altre connessioni in ingresso. Le porte sono così suddivise:

- **0 - 1023**: Porte *riservate* (Well-Known Ports) per i servizi di sistema standard.
- **1024 - 5000**: Porte *effimere*, assegnate dinamicamente dal sistema operativo al client e con durata breve.
- **5001 - 65535**: Porte *libere*, utilizzabili per scopi generali o applicazioni custom.

5.2 Primitive di Comunicazione (Berkeley UNIX)

La chiamata alla funzione nativa `socket()` restituisce sempre un *descrittore*. In generale, per instaurare una comunicazione si usano funzioni logiche come `listen`, `connect`, `send`, `receive` e `disconnect`.

5.2.1 Primitive TCP

Per le connessioni Stream (TCP), il set di primitive standard Berkeley prevede:

- `socket`: Crea l'endpoint di comunicazione.
- `bind`: Associa (lega) l'indirizzo IP locale e la porta al descrittore della socket (usata tipicamente dal Server).
- `listen`: Prepara il Server ad accettare connessioni in ingresso, mettendolo in ascolto.
- `accept`: Accetta una richiesta di connessione da un client (blocca il processo finché non arriva una richiesta).
- `connect`: Utilizzata dal Client, specificando l'indirizzo del destinatario, per stabilire la connessione con il Server.
- `send` / `recv`: Invia e riceve i dati.
- `close`: Chiude definitivamente la connessione.

5.2.2 Primitive UDP

Per le connessioni Datagram (UDP), non essendoci una connessione permanente, il flusso è più semplice (gli appunti specificano che non viene usata la `bind`):

- `sendto`: Invia i dati specificando esplicitamente, ogni volta, l'indirizzo del destinatario.
- `recvfrom`: Riceve i dati leggendo anche il mittente da cui provengono.

5.3 Strutture Dati e Funzioni in C

Per gestire gli indirizzi IP e le porte, le librerie di rete in C utilizzano specifiche `struct`.

```
struct in_addr {
    in_addr_t s_addr;       // Indirizzo IPv4 a 32 bit
};

struct sockaddr_in {
    uint8_t sin_len;        // Lunghezza della struttura
    sa_family_t sin_family; // AF_INET (Indica la famiglia IPv4)
    in_port_t sin_port;     // Numero di porta
    struct in_addr sin_addr;// Indirizzo IPv4
    char sin_zero[8];       // Inutilizzato: inizializzare sempre a 0!
};
```

5.3.1 Funzioni di Utilità e Conversione

- `gethostname(char *hostname, int len)`: Richiede il nome dell'host su cui sta girando il programma.
- `inet_addr(char *address)`: Converte una stringa IP in notazione puntata (es. "192.168.1.1") in un valore numerico a 32 bit.
- `inet_ntoa(struct in_addr address)`: Fa l'operazione opposta, convertendo la struttura a 32 bit in una stringa puntata.
- `gethostbyname(const char *hostname)`: Restituisce una struttura `hostent` con le informazioni dell'host (es. risoluzione DNS di www.google.it).
- `gethostbyaddr(const void *addr, int len, int type)`: Restituisce il nome dell'host a partire dal suo indirizzo IP (Reverse DNS lookup).

5.4 Interazione Hardware: Il Network Adaptor

La Scheda di Rete (*Network Adaptor*) funge da interfaccia fisica tra l'Host (CPU/Memoria) e la Rete. È costituita da due sezioni separate che comunicano tra loro tramite una **FIFO**:

1. Una parte interagisce con la CPU dell'Host.
2. L'altra parte interagisce con la rete esterna implementando il livello *Fisico* e di *Collegamento* (Data Link).

Il sistema è gestito da un **SCO** (Sottosistema di Controllo della Scheda) che comunica con la CPU tramite il **CSR** (*Control and Status Register*), un registro hardware in cui entrambi possono leggere e settare dei *flag* (bit di stato). L'Host può monitorare il CSR per sapere cosa accade sulla scheda in due modi:

- **Busy waiting**: La CPU legge ciclicamente il CSR finché non rileva una modifica. È uno spreco di cicli CPU per un PC normale, ma è ragionevole per dispositivi dedicati esclusivamente all'instradamento di pacchetti, come i router.
- **Interrupt**: Il Network Adaptor invia un segnale hardware (interrupt) all'Host, il quale interrompe temporaneamente le sue attività per andare a leggere il CSR e gestire l'evento.

5.4.1 Trasferimento Dati (Memoria ↔ Adaptor)

Per spostare materialmente i pacchetti (Frame) tra la RAM dell'Host e la scheda di rete, esistono due approcci:

- **Programmed I/O**: Lo scambio dei dati passa direttamente attraverso i registri della CPU. È necessario bufferizzare almeno un frame sulla CPU, e la memoria RAM deve essere *dual port* affinché processore e adaptor possano leggere e scrivere.
- **Direct Memory Access (DMA)**: Molto più efficiente, non coinvolge la CPU nel trasferimento. L'Host preassegna un'area di memoria specifica in cui l'Adaptor andrà a scrivere o leggere direttamente i frame. In questo modo, alla scheda servono pochissimi byte di buffer a bordo.

6 Sistemi Distribuiti e IoT

Un sistema distribuito è un insieme di entità (processi e thread) separate spazialmente, ovvero in esecuzione su computer differenti, che comunicano e si coordinano per apparire all'utente finale come un unico sistema coerente.

6.1 Perché usare i Sistemi Distribuiti?

I sistemi distribuiti (es. LAN, DBMS, ATM, Internet, Cloud Computing) sono spesso preferibili a quelli centralizzati per vari motivi:

- **Economicità**: Più sistemi piccoli interconnessi sono spesso più economici di un singolo mainframe di pari potenza.
- **Velocità**: Permettono di aggregare una potenza di calcolo altrimenti inottenibile.
- **Affidabilità e Tolleranza ai guasti**: Il guasto di un singolo nodo non fa crollare l'intero sistema (resilienza).
- **Scalabilità**: È possibile espandere il sistema in modo incrementale aggiungendo semplicemente nuovi nodi.

6.1.1 Sfide e Problematiche

Gestire un sistema distribuito comporta ostacoli significativi:

- **Eterogeneità**: Macchine, reti, SO e linguaggi di programmazione diversi devono poter comunicare.
- **Apertura e Flessibilità**: Il sistema deve fornire interfacce pubbliche e standard (es. il Web) ed essere facilmente modificabile.
- **Concorrenza e Sincronizzazione**: La concorrenza non è più solo temporale ma spaziale. Manca un clock globale e le latenze di rete sono imprevedibili.
- **Trasparenza**: L'utente non deve accorgersi che i dati sono sparsi su macchine diverse.

6.2 Architetture: Client-Server, Peer-to-Peer e SOA

L'inizializzazione e la struttura dei sistemi distribuiti si basano spesso su modelli:

- **Client-Server**: Un client (generalmente con minori prestazioni) inoltra richieste a un server che esegue le computazioni pesanti. Si possono dividere le responsabilità su vari livelli (*Presentation logic, Application logic, Database logic*).
- **Service Oriented Architecture (SOA)**: Struttura modulare in cui i *Service Provider* registrano i loro servizi presso un *Service Broker*. I *Service Requester* chiedono al broker a chi rivolgersi per ottenere una specifica funzione.

6.2.1 Il Middleware e le RPC

Per risolvere l'eterogeneità si inserisce un livello software chiamato **Middleware** tra il SO e le applicazioni. Il meccanismo principale di comunicazione sono le **Remote Procedure Call (RPC)**, che permettono di invocare una funzione situata su un server remoto come se fosse una funzione locale, mascherando lo scambio di messaggi di rete (tramite *stub programs*). Se si verificano problemi di connessione, le RPC applicano diverse politiche di risoluzione: *At least once* (riprova finché non va, rischio di duplicati), *At most once* (prova una volta e poi rinuncia) o *Exactly once* (la più sicura ma complessa da implementare).

6.3 Internet of Things (IoT)

L'IoT comprende qualsiasi dispositivo (es. smartphone, sensori, smart TV) dotato di capacità di calcolo, sensori, attuatori e di un'identificazione univoca in rete, capace di scambiare dati. Poiché questi dispositivi hanno spesso RAM/ROM estremamente limitate e devono consumare poca energia (low-power), i Sistemi Operativi IoT devono essere:

- Altamente ottimizzati e con footprint di memoria ridottissimo.
- Efficienti dal punto di vista energetico.
- Sicuri e con capacità di risposta *real-time*.

7 Sicurezza Informatica

La sicurezza informatica studia i meccanismi con cui si proteggono sistemi, dati e comunicazioni da accessi non autorizzati, modifiche indesiderate, interruzioni del servizio e comportamenti malevoli. Nel contesto dei sistemi di calcolo, la sicurezza non riguarda soltanto la difesa da attacchi esterni, ma anche la corretta progettazione del software, la gestione degli utenti, la protezione delle risorse di sistema e il controllo delle informazioni che circolano in rete.

7.1 Obiettivi della Sicurezza

Gli obiettivi fondamentali della sicurezza sono tradizionalmente riassunti nella **triade CIA**:

- **Confidentiality (Confidenzialità)**: garantisce che i dati siano accessibili solo ai soggetti autorizzati.
- **Integrity (Integrità)**: garantisce che i dati non vengano alterati in modo illecito o accidentale.
- **Availability (Disponibilità)**: garantisce che i servizi e le informazioni siano accessibili quando necessario.

A questi tre obiettivi si aggiungono spesso altre proprietà importanti:

- **Authenticity**: consente di verificare l'identità di un utente, di un processo o del mittente di un messaggio.
- **Accountability**: permette di ricostruire chi ha compiuto una certa azione grazie a meccanismi di tracciamento, logging e audit.
- **Non ripudio**: impedisce a un soggetto di negare di aver compiuto una determinata operazione, ad esempio l'invio di un messaggio firmato digitalmente.

7.2 Minacce, Vulnerabilità e Attacchi

Per comprendere la sicurezza è utile distinguere tre concetti:

- **Minaccia**: un possibile evento dannoso per il sistema.
- **Vulnerabilità**: una debolezza del sistema che può essere sfruttata.
- **Attacco**: l'azione concreta con cui una vulnerabilità viene sfruttata.

Gli attacchi possono essere classificati in due grandi categorie:

- **Attacchi passivi**: l'attaccante osserva il sistema senza modificarlo, per esempio intercettando traffico di rete o leggendo informazioni riservate.
- **Attacchi attivi**: l'attaccante modifica dati, interferisce con il funzionamento del sistema o genera traffico malevolo.

Tra gli attacchi passivi rientrano, ad esempio, l'intercettazione di comunicazioni e l'analisi del traffico. Tra gli attacchi attivi rientrano invece la modifica dei messaggi, il furto di identità, il replay di dati precedentemente registrati e i tentativi di negare il servizio agli utenti legittimi.

7.3 Malware

Con il termine **malware** si indica qualsiasi software progettato con finalità dannose o comunque ostili al proprietario del sistema. Il malware può mirare al furto di dati, al controllo remoto della macchina, alla distruzione di informazioni, alla raccolta di credenziali o all'interruzione del servizio.

Tra le principali categorie troviamo:

- **Trojan Horse**: programma apparentemente legittimo che nasconde codice malevolo.
- **Backdoor**: meccanismo nascosto che consente di aggirare i normali controlli di autenticazione.
- **Rootkit**: insieme di strumenti usati per ottenere privilegi elevati e nascondere la presenza dell'attaccante nel sistema.
- **Bot**: software che permette il controllo remoto di una macchina compromessa.
- **Botnet**: rete di macchine infette controllate da un attaccante in modo coordinato.

Le botnet sono particolarmente pericolose perché possono essere utilizzate per lanciare attacchi distribuiti su larga scala, come i **DDoS** (*Distributed Denial of Service*), nei quali molte macchine inviano contemporaneamente richieste verso un server bersaglio per saturarne risorse, banda o capacità di elaborazione.

Il malware rappresenta una minaccia grave anche perché spesso sfrutta i privilegi dell'utente che lo esegue. Di conseguenza, un sistema di controllo degli accessi troppo permissivo aumenta notevolmente l'impatto di un'infezione.

7.4 Denial of Service e DDoS

Un attacco di **Denial of Service (DoS)** ha l'obiettivo di rendere un servizio indisponibile agli utenti legittimi. Questo può avvenire saturando la CPU, la memoria, la banda di rete o il numero massimo di connessioni accettabili da un server.

Nel caso del **Distributed Denial of Service (DDoS)**, l'attacco non proviene da una sola macchina ma da un insieme di dispositivi compromessi distribuiti in rete. Questo rende la difesa più difficile, perché il traffico malevolo appare distribuito e può sembrare simile a traffico legittimo.

Tra gli esempi classici rientrano:

- **TCP SYN flood**: l'attaccante apre un gran numero di richieste di connessione senza completarle, saturando le strutture del server dedicate alle connessioni pendenti.
- **Ping flood**: invio massiccio di pacchetti ICMP per congestionare la rete o il nodo bersaglio.

7.5 Il problema del Buffer Overflow

Una delle vulnerabilità classiche nei sistemi scritti in linguaggi a basso livello, come il C, è il **buffer overflow**. Il problema nasce quando un programma scrive in un buffer più dati di quanti il buffer possa contenere.

In memoria, specialmente nello stack, le variabili locali, i parametri e l'indirizzo di ritorno di una funzione possono trovarsi in locazioni adiacenti. Se un buffer viene riempito oltre i suoi limiti, i dati in eccesso possono sovrascrivere altre informazioni critiche, inclusi puntatori e *return address*.

Questo consente, in alcuni casi, a un attaccante di:

1. inserire del codice malevolo in memoria;
2. modificare il return address;
3. forzare il processore a eseguire il codice scelto dall'attaccante al termine della funzione.

Il buffer overflow è quindi un esempio perfetto di vulnerabilità che nasce da una cattiva gestione della memoria e che può portare all'esecuzione arbitraria di codice.

7.5.1 Contromisure al Buffer Overflow

Le principali difese contro questo tipo di attacco sono:

- **Controllo dei limiti**: il compilatore o il programmatore devono verificare che gli array non vengano scritti oltre la loro dimensione.
- **Linguaggi memory-safe**: alcuni linguaggi moderni impediscono per costruzione l'accesso fuori dai limiti.
- **Canaries**: valori speciali inseriti nello stack prima del return address; se vengono alterati, il programma rileva l'attacco e termina.
- **DEP (Data Execution Prevention)**: impedisce l'esecuzione di codice in aree di memoria destinate soltanto ai dati, come lo stack.

Queste difese non eliminano tutti i problemi di sicurezza, ma rendono molto più difficile sfruttare una vulnerabilità di overflow.

7.6 Autenticazione e Gestione delle Password

La sicurezza di un sistema dipende in larga parte dalla capacità di riconoscere correttamente gli utenti che vi accedono. Questo processo prende il nome di **autenticazione**.

I metodi di autenticazione si basano tipicamente su uno o più dei seguenti fattori:

- qualcosa che l'utente **conosce**, ad esempio una password;
- qualcosa che l'utente **possiede**, ad esempio un token o uno smartphone;
- qualcosa che l'utente **è**, ad esempio un dato biometrico.

Nel caso delle password, una buona pratica fondamentale consiste nel non salvarle mai in chiaro. Il sistema salva invece il risultato di una funzione di **hash**, cioè una trasformazione unidirezionale della password.

Per aumentare la sicurezza, all'hash si aggiunge un valore casuale detto **salt**. Il salt fa sì che due utenti con la stessa password non abbiano lo stesso hash memorizzato e rende molto meno efficaci gli attacchi a dizionario e le tabelle precalcolate.

In generale, la robustezza di un sistema di autenticazione non dipende solo dalla complessità della password, ma anche dal modo in cui viene archiviata, verificata e protetta nel tempo.

7.7 Crittografia

La **crittografia** è l'insieme delle tecniche con cui si trasformano i dati in modo da renderli comprensibili solo ai soggetti autorizzati. Il messaggio originale è detto *plaintext*, mentre il messaggio cifrato è detto *ciphertext*.

7.7.1 Crittografia Simmetrica

Nella **crittografia simmetrica** si utilizza la stessa chiave sia per cifrare sia per decifrare. Questo approccio è molto efficiente dal punto di vista computazionale, ma presenta un problema fondamentale: la chiave deve essere condivisa in modo sicuro tra le parti prima che inizi la comunicazione.

Gli algoritmi simmetrici sono tipicamente usati per:

- cifrare grandi quantità di dati in modo veloce;
- proteggere file o flussi di comunicazione una volta che la chiave è stata scambiata;
- lavorare su blocchi di dati o su stream continui.

7.7.2 Crittografia Asimmetrica

Nella **crittografia asimmetrica** si utilizza una coppia di chiavi:

- una **chiave pubblica**, che può essere distribuita liberamente;
- una **chiave privata**, che deve rimanere segreta.

Se si cifra un messaggio con la chiave pubblica del destinatario, solo la corrispondente chiave privata potrà decifrarlo. Questo risolve elegantemente il problema della distribuzione delle chiavi, ma il costo computazionale è maggiore rispetto alla crittografia simmetrica.

Tra gli algoritmi asimmetrici più noti c'è **RSA**. In pratica, molti protocolli moderni combinano i due approcci: si usa la crittografia asimmetrica per scambiare in sicurezza una chiave simmetrica temporanea, e poi si usa la crittografia simmetrica per i dati veri e propri.

7.8 Integrità dei messaggi

Proteggere la sola confidenzialità non basta. In molti casi è necessario verificare anche che il messaggio non sia stato alterato durante il trasporto.

A questo scopo si usano funzioni di hash e meccanismi di autenticazione del messaggio. Un **MAC** (*Message Authentication Code*, da non confondere con Mandatory Access Control) permette di verificare sia l'integrità sia l'autenticità di un messaggio condividendo una chiave segreta tra mittente e destinatario.

L'idea generale è che due messaggi diversi non dovrebbero produrre lo stesso valore di controllo, e che un attaccante non dovrebbe riuscire a generare un codice valido senza conoscere la chiave.

7.9 Controllo degli Accessi

Il **controllo degli accessi** stabilisce quali soggetti possono accedere a quali risorse e con quali permessi. I permessi tipici riguardano operazioni come lettura, scrittura, modifica, esecuzione o amministrazione.

In un sistema sicuro non basta autenticare un utente: bisogna anche decidere che cosa quell'utente può effettivamente fare. Questa decisione può essere implementata attraverso diversi modelli.

7.9.1 DAC – Discretionary Access Control

Nel modello **DAC**, il controllo degli accessi è lasciato alla discrezione del proprietario della risorsa. Chi crea un file o una risorsa può decidere quali altri utenti autorizzare e con quali diritti.

Questo modello è molto flessibile ed è tipico di molti sistemi operativi general-purpose. Ad esempio, nei permessi dei file in Unix o Windows, il proprietario di un oggetto può spesso modificarne i permessi e condividerlo con altri utenti.

Il vantaggio del DAC è la semplicità e la facilità d'uso. Lo svantaggio è che il proprietario può trasferire i permessi a terzi, volontariamente o involontariamente, e questo rende il modello meno rigoroso dal punto di vista della sicurezza.

Se un account utente viene compromesso, il malware eredita spesso gli stessi diritti dell'utente. Per questo il DAC è comodo e diffuso, ma non è il modello più adatto agli ambienti ad alta sicurezza.

7.9.2 MAC – Mandatory Access Control

Nel modello **MAC**, il controllo degli accessi non è deciso dal proprietario della risorsa ma da una politica centrale imposta dal sistema. Utenti e dati vengono associati a livelli o etichette di sicurezza, e l'accesso è concesso solo se la politica lo permette.

Un esempio classico è l'ambiente militare, in cui i documenti possono essere classificati come *riservato*, *segreto* o *top secret*. Un utente con autorizzazione insufficiente non può leggere un documento classificato a un livello superiore, anche se il documento è stato creato da lui o da un collega.

Il vantaggio principale del MAC è l'elevato livello di protezione. Lo svantaggio è la rigidità: il sistema è più difficile da amministrare, meno flessibile e spesso meno adatto agli ambienti collaborativi. Per questo motivo il MAC viene usato soprattutto in contesti governativi, militari o ad altissima criticità.

7.9.3 RBAC – Role-Based Access Control

Nel modello **RBAC**, i permessi non vengono assegnati direttamente ai singoli utenti ma ai **ruoli**. Gli utenti vengono poi associati a uno o più ruoli, e ottengono automaticamente i permessi previsti da quei ruoli.

Ad esempio, un utente può appartenere al ruolo *studente*, *docente*, *amministratore* o *tecnico di laboratorio*. Ogni ruolo possiede un insieme di autorizzazioni coerente con la funzione che svolge nel sistema.

L'RBAC è molto usato in aziende, università e infrastrutture organizzate, perché semplifica enormemente la gestione. Se una persona cambia mansione, spesso è sufficiente cambiare ruolo per aggiornare tutti i permessi in modo coerente.

Questo modello si lega bene al **principio del minimo privilegio**, secondo cui ogni utente dovrebbe possedere solo i permessi strettamente necessari per svolgere il proprio compito. Il principale limite dell'RBAC è che richiede una progettazione iniziale accurata dei ruoli; se i ruoli vengono definiti male, il sistema diventa difficile da mantenere.

7.9.4 Confronto tra DAC, MAC e RBAC

Si possono riassumere le differenze principali nel seguente modo:

- **DAC**: modello flessibile, centrato sul proprietario della risorsa.
- **MAC**: modello rigido, centrato su etichette e classificazioni di sicurezza.
- **RBAC**: modello organizzativo, centrato sui ruoli e sulle responsabilità.

In generale:

- il **DAC** privilegia la facilità d'uso;
- il **MAC** privilegia la massima sicurezza;
- il **RBAC** privilegia la scalabilità amministrativa.

7.10 Firewall

Un **firewall** è un meccanismo di protezione che controlla il traffico di rete in ingresso e in uscita applicando una politica di sicurezza. Il suo obiettivo è impedire comunicazioni non autorizzate tra una rete fidata e reti non fidate.

Un firewall può filtrare il traffico sulla base di:

- indirizzi IP sorgente e destinazione;
- porte sorgente e destinazione;
- protocollo utilizzato;
- stato della connessione;
- applicazione che genera il traffico.

In pratica, il firewall realizza una barriera logica che lascia passare solo il traffico considerato legittimo dalla politica di sicurezza definita dall'amministratore. Naturalmente, da solo non basta a proteggere l'intero sistema, ma è uno degli strumenti fondamentali della difesa perimetrale.

7.11 Honeypot

Un **honeypot** è un sistema-esca progettato per apparire interessante o vulnerabile agli occhi di un attaccante. Lo scopo non è offrire un vero servizio produttivo, ma attirare l'attacco in un ambiente controllato.

Gli honeypot vengono usati per:

- studiare le tecniche degli attaccanti;
- raccogliere indicatori di compromissione;
- rilevare attività anomale;
- deviare l'attenzione da sistemi realmente critici.

Dal punto di vista didattico, sono utili perché mostrano che la sicurezza non consiste solo nel bloccare gli attacchi, ma anche nel capire come si comporta l'avversario.

7.12 Principi generali di progettazione sicura

Al di là dei singoli strumenti, un sistema sicuro dovrebbe essere progettato secondo alcuni principi generali:

- **Minimo privilegio**: ogni utente o processo deve avere solo i permessi strettamente necessari.
- **Difesa in profondità**: non si deve dipendere da un unico meccanismo di sicurezza.
- **Fail-safe default**: in caso di dubbio, l'accesso deve essere negato.
- **Riduzione della superficie d'attacco**: meno servizi inutili, meno porte aperte, meno codice esposto.
- **Aggiornamento e monitoraggio**: la sicurezza non è statica, ma richiede patch, log e verifiche continue.

Questi principi collegano tutti gli argomenti del capitolo: autenticazione, autorizzazione, crittografia, protezione della memoria, difesa di rete e monitoraggio. Una buona sicurezza non deriva da una sola tecnica, ma dalla combinazione coerente di più livelli di protezione.

8 Algoritmo di Dekker (2 Processi)

L'obiettivo è garantire la mutua esclusione per 2 processi (`P0` e `P1`) senza alcun supporto hardware. Esaminiamo i vari tentativi per capire gli errori concettuali.

Primo Tentativo: Variabile Turno

Uso di una singola variabile globale `turn`.

```
global int turn = 0; // P0 ha id 0, P1 ha id 1
while (true) {
    /* Non-Critical Section (NCS) */
    while (turn != me) { /* busy wait */ }
    /* Critical Section (CS) */
    turn = other;
}
```

Problema: Quando non è il mio turno, sto in *busy waiting*. All'atto pratico, i due processi sono vincolati a una rigida alternanza e viaggiano alla velocità del più lento.

Secondo Tentativo: Due Flag

Invece del turno, uso due flag per indicare se l'altro processo è in CS.

```
global boolean flag[2] = {false, false};
while (true) {
    /* NCS */
    while (flag[other]) { /* busy wait */ }
    flag[me] = true;
    /* CS */
    flag[me] = false;
}
```

Problema: Questa soluzione non garantisce la mutua esclusione. Entrambi potrebbero superare il ciclo `while` simultaneamente e settare il proprio flag a `true`, entrando entrambi in CS.

Terzo Tentativo: Imposto prima il Flag

Adesso imposto il flag prima del controllo.

```
while (true) {
    /* NCS */
    flag[me] = true;
    while (flag[other]) { /* busy wait */ }
    /* CS */
    flag[me] = false;
}
```

Problema: Si crea **Deadlock**. Se i due processi impostano il flag a `true` contemporaneamente, rimarranno bloccati a vicenda nel `while` per sempre.

Quarto Tentativo: Ritirata attiva (Delay)

Dichiaro di voler entrare, controllo l'altro, e se vuole entrare anche lui ritiro la mia richiesta, aspetto e riprovo.

```
while (true) {
    /* NCS */
    flag[me] = true;
    while (flag[other]) {
        flag[me] = false;
        /* delay */
        flag[me] = true;
    }
    /* CS */
    flag[me] = false;
}
```

Problema: Funziona quasi sempre, ma è poco elegante. Esiste il rischio di *Livelock*, dove entrambi ritirano e ripropongono il flag all'infinito in perfetta sincronia.

8.1 Soluzione Definitiva (Dekker)

La soluzione unisce l'idea dei Flag per l'intenzione con la variabile Turno per risolvere i conflitti simultanei a favore di chi aspetta da più tempo.

```
int me = 0, other = 1;
while (true) {
    /* NCS */
    flag[me] = true; // Dichiaro di voler entrare
```

```
    while (flag[other]) { // Se anche l'altro vuole entrare
        if (turn == other) { // Guardo il turno
            flag[me] = false; // Rilascio l'intenzione
            while (turn == other) { /* busy wait */ }
            flag[me] = true; // Dichiaro di nuovo di voler entrare
        }
    }
    /* CS */
    turn = other; // Sei il prossimo a entrare
    flag[me] = false;
}
```

9 Algoritmo di Dijkstra (Generalizzazione a N processi)

Dijkstra estende il concetto a un numero `N` di processi, usando array condivisi e un turno `k`.

```
global boolean interested[N] = {false, ..., false};
global boolean passed[N] = {false, ..., false};
int k = 0; // Turno (0 <= k < N)

// Per il processo i:
while (true) {
    /* NCS */
1.  interested[i] = true; // Dichiaro interesse
2.  while (k != i) { // Check Turno
3.      passed[i] = false;
4.      if (!interested[k]) { k = i; } // Se chi ha il turno non lo vuole, me lo rubo
    }
5.  passed[i] = true; // Sono passato
6.  for (j=1; j<=N; j except i) { // Controllo gli altri
7.      if (passed[j]) goto 2; // Se qualcuno e' gia' passato, ricomincio
    }
8.  /* Critical Section */
9.  passed[i] = false;
    interested[i] = false; // Libera tutto
}
```

Note su Dijkstra: Garantisce la **Mutua Esclusione**, ma **NON garantisce l'assenza di Starvation**. Tutto gira in un ciclo infinito: un processo, una volta finito, potrebbe rimettersi subito in coda ed essere favorito, lasciando un processo "lento" in attesa perenne.

10 Algoritmo del Panettiere (Lamport's Bakery)

Non si fa a gara per un turno globale: ogni processo estrae un "numeretto" e si aspetta in fila, proprio come in panetteria.

Implementazione Base (Senza ordinamento)

```
while (1) {
    /* NCS */
    number[i] = 1 + max(number[1], ..., number[N]); // Calcolo il numeretto
    for (j=1; j<=N; j except i) {
        while (number[j] != 0 && number[j] < number[i]) { /* wait */ }
    }
    /* CS */
    number[i] = 0;
}
```

Problema: **Deadlock**. Se due processi calcolano il numero in contemporanea e ottengono lo stesso numeretto, si bloccheranno a vicenda.

Implementazione 2 (Ordine Lessicografico)

Per risolvere i numeri doppi, si usa l'ID del processo. Se hanno lo stesso numero, entra quello con ID minore. Ordine: `(number, ID)`. `(A, i) < (B, j)` se `A < B`, oppure se `A == B` e `i <`

j. **Problema residuo**: Se mentre un processo sta facendo i controlli, l'altro riesce a prendere un numero (magari identico al mio ma con ID minore), perdo la mutua esclusione. Le variabili devono essere protette durante la fase di "calcolo".

10.1 La Soluzione Definitiva di Lamport

Introduciamo l'array `choosing` (booleano). Segnala agli altri: "Ehi, sto calcolando il mio numero, non fare confronti con me finché non ho finito".

```
while (1) {
    /* NCS */
    choosing[i] = true; // Inizio estrazione numero
    number[i] = 1 + max(number[1], ..., number[N]);
    choosing[i] = false; // Fine estrazione

    for (j=1; j<=N; j except i) {
        while (choosing[j] == true) { /* wait */ } // Aspetto che
            l'altro estragga
        // Confronto lessicografico:
        while (number[j] != 0 && (number[j], j) < (number[i], i)) { /*
            wait */ }
    }
    /* CS */
    number[i] = 0;
}
```

Calcolo del Massimo in memoria locale

Per evitare problemi di scrittura, il massimo si calcola atomicamente usando variabili locali al processo:

```
int local1 = 0;
for (int j=1; j<=N; j++) {
    int local3 = number[j];
    if (local1 < local3) { local1 = local3; }
}
number[i] = 1 + local1;
```

Il problema dell'Overflow

Problema: Se c'è sempre almeno un processo che vuole entrare in CS, il numeretto aumenterà all'infinito causando **overflow** della variabile. **Soluzione (*Bounded Numbers*)**: Impostare un tetto massimo `MAXIMUM`. Quando il calcolo supera il massimo (`number % MAXIMUM == 0`), il processo aspetta (*busy waiting*) finché tutti quelli che hanno già il numero non finiscono, per poi estrarre un numero da zero.

Riassunto Bakery: Garantisce la **Mutua Esclusione** e la **No Starvation**. Tuttavia, **NON è una coda FIFO perfetta**, poiché se due processi arrivano contemporaneamente, l'ordine di ingresso è dettato dall'ID di macchina e non dall'effettivo tempo di arrivo.